

Compiladores 2026–2

Práctica 0

Sistema de procesamiento de Lenguaje

Prof. Ariel Adara Mercado Martínez

Ayud. Carlos Gerardo Acosta Hernández

Adud. Luis Mario Escobar Rosales

Rodrigo André Decuir Fuentes

No. Cuenta: 320022711

[ojo@ciencias.unam.mx](mailto:ojo@ciencias.unam.mx)

Fecha de entrega: 27 de febrero



## Preguntas

Use el siguiente comando: `cpp programa.c programa.i`

Revise el contenido de *programa.i* y conteste lo siguiente:

- ¿Qué ocurre cuando se invoca el comando *cpp* con esos argumentos? *cpp* es el preprocesador de c, por lo que al ejecutar el comando con esos argumentos se lee *programa.c*, se preprocesa y se escribe el programa fuente modificado en el archivo *programa.i*. La extensión *.i* se usa para código que no debe de ser preprocesado (en este caso *programa.i* ya no debe de ser preprocesado porque básicamente es el contenido preprocesado de *programa.c*).
- ¿Qué similitudes encuentra entre los archivos *programa.c* y *programa.i*? *programa.i* cuenta con todo lo que existía en *programa.c* y más, pues en el archivo preprocesado (*programa.i*) ya se cuenta con las macros expandidas, el reemplazo de los comentarios por espacios en blanco, entre otras fases que forman parte de la traducción del preprocesador.
- ¿Qué pasa con las macros y los comentarios del código fuente original en *programa.i*? Los comentarios se reemplazan por espacios en blanco y las macros se expanden, es decir, se reemplaza su llamada por la definición completa correspondiente.
- Compare el contenido de *programa.i* con el de *stdio.h* e indique de forma general las similitudes entre ambos archivos. Como *stdio.h* es un header que se incluyó en *programa.c* vamos a tener que todo su contenido prácticamente se copia y pega a *programa.i*, de este modo, como similitud podríamos decir que uno contiene al otro.
- ¿A qué etapa corresponde este proceso? Al programa fuente modificado. Justo después de pasar por el preprocesador.

Ejecute la siguiente instrucción: `gcc -Wall -S programa.i`

- ¿Para qué sirve la opción *-Wall*? Para habilitar todas las advertencias que vienen dadas por escribir código de forma cuestionable y que usualmente son fáciles de modificar para prevenir que se lance una advertencia. Asimismo, se revisan las macros y se lanza una advertencia en caso de que alguna se encuentre escrita de forma cuestionable.
- ¿Qué le indica a gcc la opción *-S*? Que gcc se detenga antes de ensamblar. Asimismo existen las opciones *-c* y *-E* para detener la compilación en otras etapas.
- ¿Qué contiene el archivo de salida y cuál es su extensión? El código traducido a lenguaje ensamblador. Su extensión es *.s*.
- ¿A qué etapa corresponde este comando? Al momento en que se obtiene el programa objeto en lenguaje ensamblador (justo antes de ensamblar).

Ejecute la siguiente instrucción: `as programa.s -o programa.o`

- Antes de revisarlo, indique cuál es su hipótesis sobre lo que debe contener el archivo con extensión *.o*. Pienso que el código máquina relocizable porque es el siguiente paso del proceso de compilación.
- Diga de forma general qué contiene el archivo *programa.o* y por qué se visualiza de esa manera. Las cadenas que se van a imprimir como resultado de ejecutar el programa, datos de mi SO y su versión, me parece que código binario y otros datos que no entiendo muy bien.

- c. ¿Qué programa se invoca con *as*? El ensamblador portable de GNU.
- d. ¿A qué etapa corresponde la llamada a este programa? La generación del código máquina relocizable.

Encuentre la ruta de los siguientes archivos en el equipo de trabajo:

- `ld-linux-x86-64.so.2` → `/lib64/ld-linux-x86-64.so.2`
- `Scrt1.o` (o bien, `crt1.o`) → `/usr/lib/gcc/x86_64-linux-gnu/13/../../../../x86_64-linux-gnu/Scrt1.o`
- `crti.o` → `/usr/lib/gcc/x86_64-linux-gnu/13/../../../../x86_64-linux-gnu/crti.o`
- `crtbeginS.o` → `/usr/lib/gcc/x86_64-linux-gnu/13/crtbeginS.o`
- `crtendS.o` → `/usr/lib/gcc/x86_64-linux-gnu/13/crtendS.o`
- `crtn.o` → `/usr/lib/gcc/x86_64-linux-gnu/13/../../../../x86_64-linux-gnu/crtn.o`

Ejecute el siguiente comando, sustituyendo las rutas que encontró en el paso anterior:

```
ld -o ejecutable -dynamic-linker /lib/ld-linux-x86-64.so.2 /usr/lib/Scrt1.o
/usr/lib/crti.o programa.o -lc /usr/lib/crtn.o
```

- a. En caso de que el comando `ld` mande errores, investigue como enlazar un programa utilizando el comando *ld*. Y proponga una posible solución para llevar a cabo este proceso con éxito.
- b. Describa el resultado obtenido al ejecutar el comando anterior. Se logró enlazar el código máquina relocizable y como resultado se obtuvo el ejecutable, lo probé y si funciona :))

Quite el comentario de la macro `#define PI` en el código fuente original y conteste lo siguiente:

- a. Genere nuevamente el archivo.i. De preferencia asigne un nuevo nombre.
- b. ¿Cambia en algo la ejecución final? En mi caso obtuve el mismo resultado en la ejecución final.

Escribe un segundo programa en lenguaje C en el que agregue 4 directivas del preprocesador de C (*cpp*). Las directivas elegidas deben jugar algún papel en el significado del programa, ser distintas entre sí y diferentes de las utilizadas en el primer programa (aunque no están prohibidas si las requieren).

```
#include <stdio.h>
#include <stdlib.h>

#define ll long long
#define max(a,b) (a<b?b:a)
#define abs(x) (x<0?(-x):x)
#define FOR(i,a,b) for (int i = (a); i < (b); ++i)

/**
```

```

* Compiladores 2026-2
*/
int main(void) {
    FOR(i,1,10) printf("-");
    printf("\nNúmero más grande entre abs(-1000000000000000000) y 3000000000000000000\n");
    FOR(i,1,10) printf("-");
    printf("\n");

    ll numero_negativo = -1000000000000000000LL;
    ll numero_positivo = 3000000000000000000LL;

    ll resultado = max(abs(numero_negativo), numero_positivo);
    printf("Resultado : %lld\n", resultado);
    return 0;
}

```

a. Explique su utilidad general y su función en particular para su programa.

ll long long  
 utilidad general: alias para acortar un tipo muy largo  
 función en particular: precisamente acortar el tipo que es bastante largo

max(a,b) (a<b?b:a)  
 utilidad general: obtener el máximo entre dos números  
 función en particular: precisamente obtener el máximo entre dos números

abs(x) (x<0?(-x):x)  
 utilidad general: convierte un número negativo en positivo  
 función en particular: precisamente tomar el valor absoluto de un número negativo

FOR(i,a,b) for (int i = (a); i < (b); ++i)  
 utilidad general: acortar el for  
 función en particular: justo para acortar el for